TESTING

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 4: Prompt for Properties and Metamorphic Relations, Not Just Concrete Assertions

- For functions where deriving a concrete expected output is difficult without re-implementing the logic, prompt the AI to generate property-based tests or metamorphic relations.
- This sidesteps the oracle problem by expressing what must always be true rather than what this specific output should be.
- Applies to sorting, encoding, mathematical transformations, and similar functions.

Guideline 4: Reasoning

- Property-based testing (PBT): express invariants that hold across all valid inputs, not just one specific input
- Example: instead of asserting sort([3,1,2]) == [1,2,3], assert the output is a permutation of the input and is non-decreasing (no oracle knowledge needed)
- LLMs can produce valid input generators and meaningful invariants when given the right prompting structure, but fall back to example-based tests without explicit instruction
- Property generation is an active research frontier; properties also help detect regression bugs that example-based tests miss

Guideline 4: Key Practices (Prompting)

- Tell the model explicitly to generate property-based tests rather than example-based tests, and name the testing framework (e.g., Hypothesis for Python)
- Ask the model to enumerate the invariants in prose first, then translate each one into a test. This lets you catch cases where a proposed invariant is not actually always true
- Useful property categories: round-trip (encode then decode returns original), idempotency, monotonicity, and metamorphic relations
- For metamorphic relations, phrase the prompt as: "Given two related inputs X and Y, what must the relationship between f(X) and f(Y) always be?"

Guideline 4: Key Practices (Reviewing the Result)

- Run generated tests with a large number of random examples (at least 500 to 1000) to get meaningful coverage of the input space
- Check that the given strategies generate the interesting inputs you care about (empty lists, single-element inputs) and add constraints if they do not
- Verify that properties are not trivially true for all inputs (e.g., assert len(result) >= 0 is vacuous)
- Combine property-based tests with specific worked examples from the specification for the common cases

Guideline 5: Generate Regression Tests from Bug Reports and Diffs

When a bug has been reported or fixed, prompt the AI assistant with the bug description or the code diff and ask it to generate a test that would have caught the bug.

Write this test before (or alongside) the fix.

Do not rely on general coverage-oriented test generation to produce tests specific enough to pin down a reported defect.

Guideline 5: Reasoning

- A regression test must fail on the buggy version and pass on the fixed version. General coverage-directed generation does not target this property
- LLMs can generate executable regression tests from bug reports alone; up to 50% of Defects4J bugs produced usable test cases
- Tests generated from natural-language bug reports serve as filters to distinguish correct from incorrect patches
- Every confirmed bug represents a gap in the test suite; the bug report or diff is the most information-dense prompt you can provide

Guideline 5: Key Practices (Prompting)

- Provide the bug description in plain language exactly as reported, including steps to reproduce and observed vs. expected behavior
- If a diff or patch is available, include the diff and ask for a test that fails on pre-patch code and passes on post-patch code
- Ask the model to add a comment explaining why the test would have caught the bug, to confirm it understood the fault correctly
- Include the instruction: "this test must FAIL before the fix is applied"

Guideline 5: Key Practices (Reviewing the Result)

- Verify the test fails on the pre-fix version of the code. Apply the test to a checkout of the unfixed version. If it passes there too, it does not guard against the regression
- After the fix is applied, verify the test passes
- Check that the test is narrow: a test failing only on the specific input from the bug report is more useful than one failing broadly on any input
- Add the test to the permanent regression suite with a comment referencing the bug report or issue number

Guideline 6: Be Skeptical of Coverage Alone, Use Mutation Testing

Do not use line coverage as the only quality signal for AI-generated tests.

Ask the model to help design tests that kill specific mutants, and run a mutation testing tool to confirm the test suite actually detects faults.

Coverage tells you which lines were executed; mutation score tells you whether the test suite would notice if those lines were changed.

Guideline 6: Reasoning

- LLMs naturally produce assertions that are syntactically plausible but semantically weak, making the coverage/fault-detection gap especially acute
- Assertion quality was often too low to catch faults even on branches that were executed
- Weak fault detection is a core open challenge; coverage metrics do not measure whether assertions are semantically correct
- Practitioners who relied purely on coverage reported a false sense of security with vacuous assertions
- Mutation score is a more reliable proxy for fault detection effectiveness than structural coverage

Guideline 6: Key Practices (Prompting)

- Ask the model to generate at least one test per mutation from a list you specify (e.g., 'write a test that would fail if the > in line 5 were changed to >=')
- Ask the model to annotate each assertion with a comment explaining which fault it is designed to detect
- For numeric computations, always request tests at values where an off-by-one mutation would produce a different result

Guideline 6: Key Practices (Reviewing the Result)

- After generating tests, run a mutation testing tool (mutmut for Python, PIT for Java) and inspect the surviving mutants
- For each surviving mutant, feed its diff back to the model and ask for a new test targeting that specific mutation
- Treat a mutation score below 70% as a signal to prompt for additional targeted tests
- Do not accept a suite just because it reports 100% line coverage; check the mutation score independently

References

[1] Wang et al. (2024). Software Testing With Large Language Models. IEEE TSE. DOI: 10.1109/TSE.2024.3368208

[2] Augusto et al. (2025). Large Language Models for Software Testing: A Research Roadmap. arXiv: 2509.25043

[3] Yuan et al. (2024). Evaluating and Improving ChatGPT for Unit Test Generation. Proc. ACM Softw. Eng., FSE, Article 76

[4] Santana et al. (2025). Software Testing with LLMs: An Interview Study with Practitioners. AIWare 2025

[5] Siddiq et al. (2024). Using LLMs to Generate JUnit Tests: An Empirical Study. EASE 2024

[6] Yang et al. (2024). On the Evaluation of LLMs in Unit Test Generation. ASE 2024

[7] Ryan et al. (2024). Code-Aware Prompting: Coverage-Guided Test Generation (SymPrompt). Proc. ACM Softw. Eng.

[8] Molina and Gorla (2024). Test Oracle Automation in the Era of LLMs. SE 2030 Workshop

[9] Barr et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE, 41(5)

References (continued)

[11] Myers et al. (2011). The Art of Software Testing, 3rd ed. Wiley. (Original work 1979)

[12] Meszaros, G. (2007). xUnit Test Patterns: Refactoring Test Code. Addison-Wesley

[13] Pan et al. (2025). ASTER: Natural and Multi-language Unit Test Generation with LLMs. ICSE 2025

[14] Lops et al. (2025). Large Language Models for Unit Test Generation: Achievements, Challenges, Road Ahead. arXiv: 2511.21382

[15] Claessen and Hughes (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. ICFP 2000

[16] Vikram et al. (2024). Can Large Language Models Write Good Property-Based Tests? arXiv: 2307.04346

[17] Plein et al. (2024). Automatic Generation of Test Cases Based on Bug Reports. ICSE 2024

[18] Kang et al. (2023). LLMs Are Few-Shot Testers: LLM-Based Bug Reproduction (Libro). ICSE 2023

[19] Zeller, A. (2009). Why Programs Fail, 2nd ed. Morgan Kaufmann

[20] Offutt and Untch (2001). Mutation 2000: Uniting the Orthogonal. Mutation 2000 Symposium